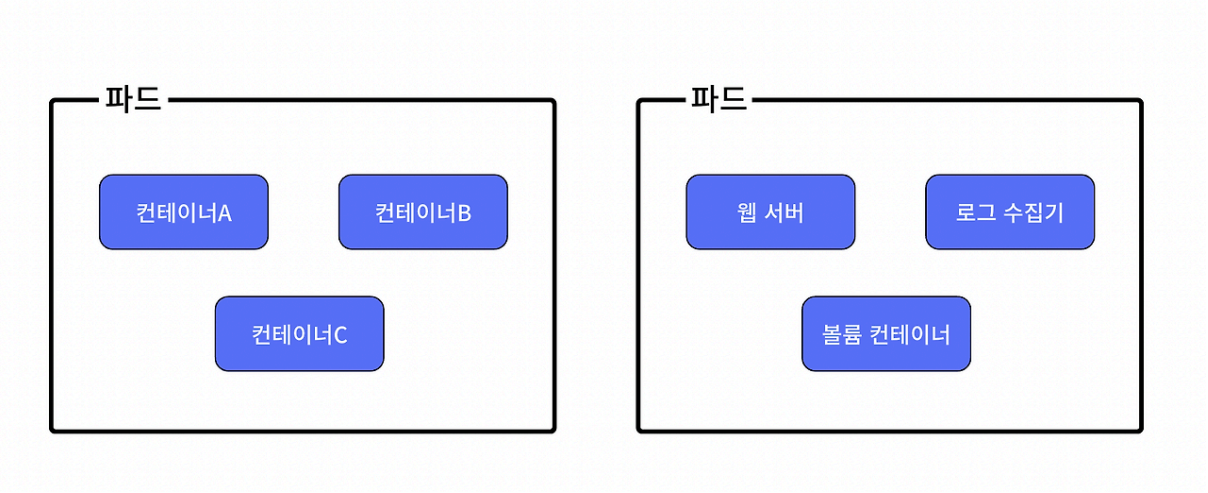
# Ch5. 파드

## 파드 개념

파드: **쿠버네티스에서 컨테이너를 관리하는 기본 단위**

- 쿠버네티스는 파드라는 단위로 **컨테이너를 묶어서 관리**하므로 보통 컨테이너 하나가 아닌 여러 개의 컨테이너로 구성될 수 있다.
- 파드 하나에 속한 컨테이너들은 모두 **노드 하나 안에서 실행**된다.




- 파드 하나 안에 있는 컨테이너들이 **IP 하나를 공유**한다.
- 파드 안 컨테이너와 통신할 때는 컨테이너마다 다르게 설정한 포트를 사용

---

## 파드 사용하기

- 파드 설정 예(pod/pod-sample.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
  labels:
    app: kubernetes-simple-pod
spec:
  containers:
```

```
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    ports:
    - containerPort: 8080
```

- .metadata.name: 파드 이름
- .metadata.labels.app: 오브젝트를 식별하는 레이블
- .spec.containers[].name: 컨테이너의 이름
  - '-'은 하위 필드를 배열 형태로 묶겠다는 뜻
- .spec.containers[].image: 컨테이너에 사용할 이미지
- .spec.containers[].ports[].containerPort: 해당 컨테이너에 접속할 포트 번호

### kubectl apply -f 메타파일명

- 메타파일 적용

```
$ kubectl apply -f pod-sample.yaml
pod/kubernetes-simple-pod created
```

### kubectl get pods

- 파드 실행 여부 확인
  - STATUS 항목이 Running이면 정상적으로 파드 실행한 것

```
$ kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
kubernetes-simple-pod   1/1     Running   0          3m3s
```

---

# 파드 생명 주기

파드는 생성부터 삭제까지의 과정에 생명주기(Lifecycle)가 있다.

- **Pending**: 파드가 생성하는 중
- **Running**: 파드 안 모든 컨테이너가 실행 중인 상태. 1개 이상의 컨테이너가 실행 중이거나 시작 또는 재시작 상태일 수 있다.
- **Succeeded**: 파드 안 모든 컨테이너가 정상 실행 종료된 상태로 재시작되지 않는다.

- **Failed**: 파드 안 모든 컨테이너 중 정상적으로 실행 종료되지 않은 컨테이너가 있는 상태
    - 컨테이너 종료 코드가 0이 아니면 비정상 종료(예: Out of memory는 종료 코드 137이거나 시스템이 직접 컨테이너를 종료한 것
- **Unknown**: 파드의 상태를 확인할 수 없는 상태

### kubectl describe pods 파드명

- Status 항목을 살펴보면 확인

```
$ kubectl describe pods kubernetes-simple-pod
Name:             kubernetes-simple-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker2/10.100.0.107
Start Time:       Sun, 29 Dec 2024 15:54:29 +0900
Labels:           app=kubernetes-simple-pod
Annotations:      <none>
Status:           Running
IP:               10.36.0.1
IPs:
  IP:  10.36.0.1
Containers:
  kubernetes-simple-pod:
    Container ID:   containerd://063f68008120e8addcc772e6a9
bd6946695e302cb50515236bfd44bf524ee422
    Image:          arisu1000/simple-container-app:latest
    ...
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
...
```

- Conditions 항목: 파드의 현재 상태 정보를 나타내며 Type과 Status로 구분되어 있다.

- Type
  - Initialized: 모든 초기화 컨테이너가 성공적으로 시작 완료되었다는 뜻
  - Ready: 파드는 요청들을 실행할 수 있고 연결된 모든 서비스의 로드밸런싱 풀에 추가되어야 한다는 뜻
  - Containers/Ready: 파드 안 모든 컨테이너가 준비 상태라는 뜻
  - PodScheduled: 파드가 하나의 노드로 스케쥴을 완료했다는 뜻
  - Unschedulable:스케줄러가 자원의 부족이나 다른 제약 등으로 지금 당장 파드를 스케쥴할 수 없다는 뜻
- Status
  - True(상태 활성화)
  - False(상태 비활성화)
  - Unknown(상태 알 수 없음)

---

# 4. kubelet으로 컨테이너 진단하기

컨테이너가 실행된 후에는 kubelet가 컨테이너를 주기적으로 진단한다.

- 이때 필요한 프로브에는 2가지가 있다.
  - **livenessProbe**
    - 컨테이너가 실행됐는지 확인
    - 이 진단이 실패하면 kubelet은 컨테이너를 종료시키고, 재시작 정책에 따라서 컨테이너를 재시작한다.
    - 컨테이너에 livenessProbe를 어떻게 할지 명시되지 않았다면 기본 상태 값은 Success이다.
  - **readinessProbe**
    - 컨테이너가 실행된 후 실제로 서비스 요청에 응답할 수 있는지 진단
    - 이 진단이 실패하면 앤드포인트 컨트롤러는 해당 파드에 연결된 모든 서비스를 대상으로 앤드포인트 정보를 제거한다.
    - 첫번째 readinessProbe를 하기 전까지의 기본 상태 값은 Failure이다.
    - readinessProbe를 지원하지 않는 컨테이너라면 기본 상태 값은 Success이다.

- 참고로 컨테이너 진단은 컨테이너가 구현한 핸들러를 kubelet이 호출해서 실행한다.
  - 핸들러 3가지
    - ExecAction: 컨테이너 안에 지정된 명령을 실행하고 종료 코드가 0일 때 Success라고 진단
    - TCPSocketAction: 컨테이너 안에 지정된 IP와 포트로 TCP 상태를 확인하고 포트가 열려있으면 Success라고 진단
    - HTTPGetAction: 컨테이너 안에 지정된 IP, 포트, 경로로 HTTP GET 요청을 보낸다. 응답 상태 코드가 200에서 400 사이면 Success라고 진단
  - 진단 결과
    - Success: 컨테이너가 진단에 성공
    - Failure: 컨테이너가 진단에 실패
    - Unknown: 진단 자체가 실패해서 컨테이너 상태를 알 수 없음

---

# 5. 초기화 컨테이너

초기화 컨테이너

- 앱 컨테이너가 실행되기 전 파드를 초기화
- 보안상 이유로 앱 컨테이너 이미지와 같이 두면 안되는 앱의 소스 코드를 별도로 관리할 때 유용
  - 초기화 컨테이너는 여러 개를 구성할 수 있다. 초기화 컨테이너가 여러 개 있다면 파드 켐플릿에 명시한 순서대로 초기화 컨테이너가 실행된다.
  - 초기화 컨테이너 실행이 실패하면 성공할 때까지 재직한다.
  - 초기화 컨테이너가 모두 실행된 후 앱 컨테이너 실행이 시작된다.
- 초기화 컨테이너는 앱 컨테이너와 비슷하게 동작하지만 몇 가지 다른 점이 있다. 그중 readinessProbe를 지원하지 않는다는 점이 중요. 파드가 모두 준비되기 전에 실행한 후 종료되는 컨테이너이기 때문
- 초기화 컨테이너 설정 예(pod/pod-init.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
```

```
  labels:
    app: kubernetes-simple-pod
spec:
  initContainers:
  - name: init-myservice
    image: arisu1000/simple-container-app:latest
    command: ['sh', '-c', 'sleep 2; echo helloworld01;']
  - name: init-mydb
    image: arisu1000/simple-container-app:latest
    command: ['sh', '-c', 'sleep 2; echo helloworld02;']
  containers:
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    command: ['sh', '-c', 'echo The app is running! && slee
p 3600']
```

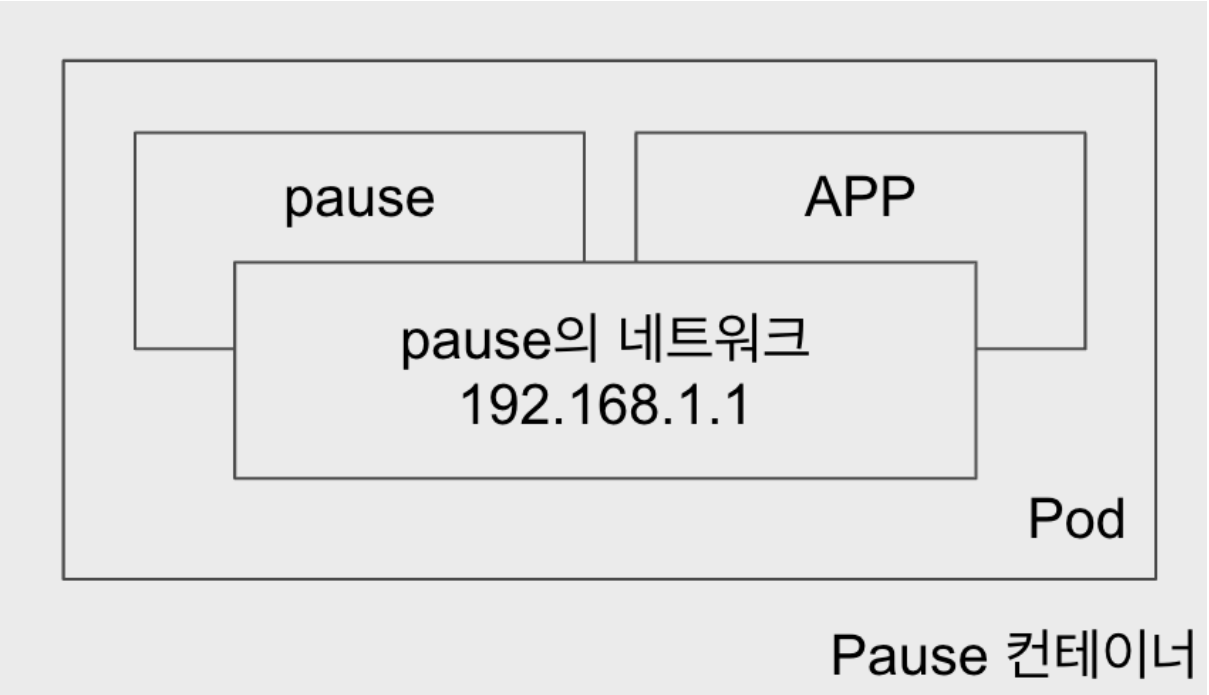
- 초기화 컨테이너를 설정하는 부분: .spec.initContainers[]의 하위 필드
  - 첫 번째 초기화 컨테이너의 .spec.initContainers[].name
  - 두 번째 초기화 컨테이너의 .spec.initContainers[].name
  - .spec.containers[]의 하위 필드는 실제로 생성할 파드
- kubernets-simple-pod라는 파드를 생성하기 전 초기화 컨테이너로 init-service와 init-mydb를 실행

```
$ kubectl apply -f pod-init.yaml
pod/kubernetes-simple-pod created
$ kubectl get pods
NAME                    READY   STATUS     RESTARTS   AGE
kubernetes-simple-pod   0/1     Init:0/2   0          5s
ubuntu@master:~/yaml$ k get pod
NAME                    READY   STATUS    RESTARTS   AGE
kubernetes-simple-pod   1/1     Running   0          34s
```

# 6. 파드 인프라 컨테이너

파드 인프라 컨테이너

- 쿠버네티스에는 모든 파드에서 항상 실행되는 pause라는 컨테이너가 있다.




- pause
  - 파드 안 기본 네트워크로 실행되며, 프로세스 식별자가 1(PID 1)로 설정되므로 다른 컨테이너의 부모 컨테이너 역할을 한다.
  - 파드 안 다른 컨테이너는 pause 컨테이너가 제공하는 네트워크를 공유해서 사용
  - 파드 안 다른 컨테이너가 재시작되었을 때 파드의 IP를 유지하지만, pause 컨테이너가 재시작되면 파드 안 모든 컨테이너도 재시작한다.
- kubelet에는 명령 옵션으로 --pod-infra-container-image가 있다. pause가 아닌 다른 컨테이너를 파드 인프라 컨테이너로 지정할 때 사용

---

# 7. Static Pod

Static Pod

- **kube-apiserver를 통하지 않고 kubelet이 직접 실행하는 파드들**
- kubelet 설정의 --pod-manifest-path라는 옵션에 지정한 디렉터리에 스태틱 파드로 실행하려는 파드들을 넣어두면 kubelet이 그걸 감지해서 파드로 실행
- 스태틱 파드는 kubelet이 직접 관리하면서 이상이 생기면 재시작
- 또한 kubelet이 실행 중인 노드에서만 실행되고 다른 노드에서는 실행되지 않는다.

- kube-apiserver로 파드를 조회할 수 있지만 스태틱 파드에 어떤 명령을 실행할 수 없다.
- 보통 스태틱 파드는 kube-apiserver라던가 etcd 같은 시스템 파드를 실행하는 용도로 많이 사용
- 스태틱 파드의 경로: **/etc/kubernetes/manifests**

---

# 8. 파드에 CPU와 메모리 자원 할당

파드에는 CPU와 메모리를 대상으로 자원 사용량을 설정하도록 .limits와 .requests 필드를 준비해두었다. 다음 4개 필드를 사용할 수 있다.

- .spec.containers[].**resources.limits.cpu**
- .spec.containers[].**resources.limits.memory**
- .spec.containers[].**resources.requests.cpu**
- .spec.containers[].**resources.requests.memory**
- 자원 사용량 설정 예(pod/pod-resource.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
  labels:
    app: kubernetes-simple-pod
spec:
  containers:
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    resources:
      requests:
        cpu: 0.1
        memory: 200M
      limits:
        cpu: 0.5
        memory: 1G
```

```
    ports:
    - containerPort: 8080
```

- requests 필드: 최소 자원 요구량
- limits 필드: 최대 자원 요구량
- 자원 사용량의 단위
  - 메모리: 바이트 단위로 측정. 별도로 용량을 나타내는 기호를 사용하지 않는다면 **기본 단위는 바이트**
  - CPU: 사용할 수 있는 자원의 코어 개수로 표시.  소수점은 어떤 뜻일까? CPU 코어 하나의 연산 능력을 기준으로 하는 것. 예를 들어 1을 설정했다면 코어 하나의 연산 능력을 온전히 사용할 수 있도록 설정했다는 뜻. 0.1로 설정했을 때는 CPU 코어 하나의 연산량을 100으로 했을 때 10%만큼 연산 능력을 할당한다는 뜻.

---

# 9. 파드에 환경 변수 설정하기

- 컨테이너를 사용할 때의 장점 중 하나는 개발 환경에서 만든 컨테이너의 환경 변수만 변경해 실제 환경에서 실행하더라도 개발 환경에서 동작하던 그대로 동작한다는 점
- 파드 환경 변수 설정 예(pod/pod-env.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
  labels:
    app: kubernetes-simple-pod
spec:
  containers:
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    ports:
    - containerPort: 8080
    env:
    - name: TESTENV01
      value: "testvalue01"
    - name: HOSTNAME
      valueFrom:
```

```
        fieldRef:
          fieldPath: spec.nodeName
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
    - name: POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP
    - name: CPU_REQUEST
      valueFrom:
        resourceFieldRef:
          containerName: kubernetes-simple-pod
          resource: requests.cpu
    - name: CPU_LIMIT
      valueFrom:
        resourceFieldRef:
          containerName: kubernetes-simple-pod
          resource: limits.cpu
```

- .spec.containers[].env[] 하위 필드
  - name: 사용할 환경 변수의 이름
  - value: 문자열이나 숫자 형식의 값
  - valueFrom: 값을 직접 할당하는 것이 아니라 어딘가 다른 곳에서 참조하는 값
  - fieldRef: 파드의 현재 설정 내용을 값으로 설정한다는 선언
  - fieldPath: .fieldRef에서 어디서 값을 가져올 것인지를 지정. 즉 값을 참조하려는 항목의 위치를 지정
  - resourceFieldRef: 컨테이너에 CPU, 메모리 사용량을 얼마나 할당했는지에 관한 정보를 가져온다.
  - containerName: 환경 변수 설정을 가져올 컨테이너 이름
  - resource: 어떤 자원의 정보를 가져올지 설정

---

# 10. 파드 환경 설정 내용 적용하기

파드는 환경 설정 내용을 템플릿 하나에 모두 작성한 후 적용해야 한다.

- 파드 환경 설정 예(pod/pod-all.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
  labels:
    app: kubernetes-simple-pod
spec:
  initContainers:
  - name: init-myservice
    image: arisu1000/simple-container-app:latest
    command: ['sh', '-c', 'sleep 2; echo helloworld01;']
  - name: init-mydb
    image: arisu1000/simple-container-app:latest
    command: ['sh', '-c', 'sleep 2; echo helloworld02;']
  containers:
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    resources:
      requests:
        cpu: 0.1
        memory: 200M
      limits:
        cpu: 0.5
        memory: 1G
    ports:
    - containerPort: 8080
    command: ['sh', '-c', 'echo The app is running! && slee
p 3600']
    env:
    - name: TESTENV01
      value: "testvalue01" # 첫 번째 환경 변수
    - name: HOSTNAME
      valueFrom:
        fieldRef:
          fieldPath: spec.nodeName # 두 번째 환경 변수
```

```yaml
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name  # 세 번째 환경 변수
    - name: POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP # 네 번째 환경 변수
    - name: CPU_REQUEST
      valueFrom:
        resourceFieldRef:
          containerName: kubernetes-simple-pod
          resource: requests.cpu # 다섯 번째 환경 변수
    - name: CPU_LIMIT
      valueFrom:
        resourceFieldRef:
          containerName: kubernetes-simple-pod
          resource: limits.cpu # 여섯 번째 환경 변수
```

- kubectl apply -f pod-all.yaml
- kubectl exec -it kubernetes-simple-pod sh
  - 제대로 환경 변수가 설정됐는지 확인
  - env 명령어를 실행해 환경변수들을 확인
  - exit 명령어를 실행해 컨테이너 외부로 나온다.

```
$ kubectl exec -it kubernetes-simple-pod sh
kubectl exec [POD] [COMMAND] is DEPRECATED and will be remo
ved in a future version. Use kubectl exec [POD] -- [COMMAN
D] instead.
Defaulted container "kubernetes-simple-pod" out of: kuberne
tes-simple-pod, init-myservice (init), init-mydb (init)
~ # env
POD_IP=10.46.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.96.0.1:443
CPU_REQUEST=1
HOSTNAME=k8s-worker1
```

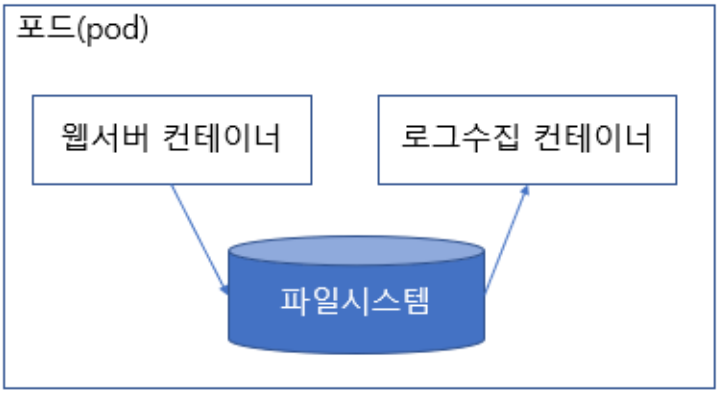
```
TESTENV01=testvalue01
SHLVL=1
...
PWD=/root
~ #
```

# 11. 파드 구성 패턴

파드를 여러 개의 컨테이너를 묶어서 구성하고 실행할 때 몇 가지 패턴을 적용할 수 있다.

## 1. 사이드카 패턴

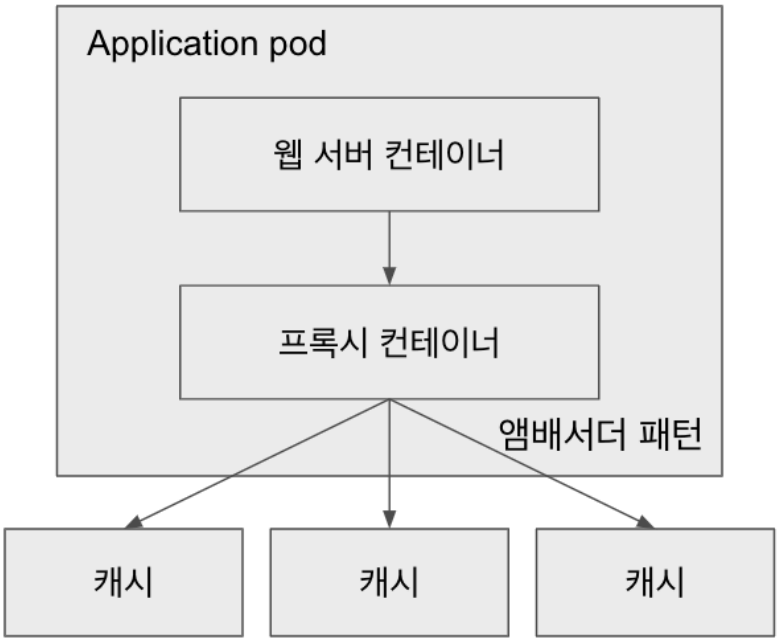
- 기본 컨테이너의 기능을 확장하거나 강화하는 용도의 컨테이너를 추가하는 것

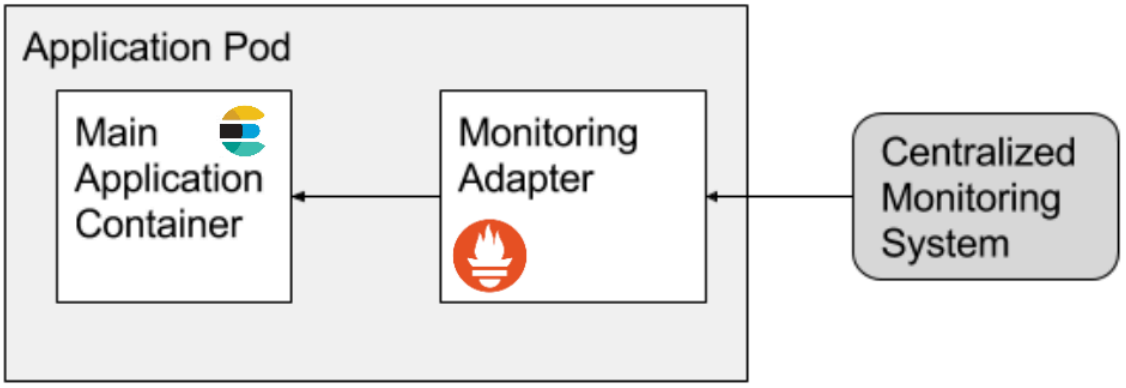



- 일반적인 웹 서버의 예라면 웹 서버 컨테이너는 웹 서버 역할만 하고 로그는 파일로 남긴다. 그럼 사이드카 역할인 로그 수집 컨테이너는 파일 시스템에 쌓이는 로그를 수집해서 외부의 로그 수집 시스템으로 보낸다.

## 2. 앰배서더 패턴

- 파드 안에서 프록시 역할을 하는 컨테이너를 추가하는 패턴
- 파드 안에서 외부 서버에 접근할 때 내부 프록시에 접근하도록 설정하고 실제 외부와의 연결은 프록시에서 알아서 처리




## 3. 어댑터 패턴

- 파드 외부로 노출되는 정보를 표준화하는 어댑터 컨테이너를 사용한다는 뜻
- 주로 어댑터 컨테이너로 파드의 모니터링 지표를 표준화한 형식으로 노출시키고, 외부의 모니터링 시스템에서 해당 데이터를 주기적으로 가져가서 모니터링하는 데 이용